스마트 저니 (Smart Journey)

현대자동차그룹 차량용 앱 개발 공모전 | 02 앱 프로젝트 소개 / 심사 자료

앱명 / 프로젝트명: 스마트 저니 (Smart Journey)

1. 앱 한 줄 소개

스마트 저니는 플레오스 내비 뒤에서 조용히 돌면서 차량 좌석 센서와 내비 정보로 맛집 원격 웨이팅을 대신 걸어 주는 차량용 앱입니다. 몇 명이 타고 있는지, 언제 도착하는지, 도착해서 몇 명이 내렸는지를 차가 확인해서 매장에 알려 주고, 운전자에게는 결정이 필요한 순간에만 승인 카드와 음성으로 나타납니다.

2. 기획 배경

2.1 주목한 순간

주말에 가족과 맛집에 갈 때 운전하는 사람은 가는 내내 두 가지를 신경 쓰게 됩니다. 언제 줄을 걸어야 할지, 그리고 몇 명으로 걸었는지와 도착 인증은 어떻게 할지입니다. 결국 운전하면서 폰을 만지게 됩니다.

- **타이밍:** 너무 일찍 걸면 도착하기 전에 호출이 와서 순번이 취소되고, 너무 늦게 걸면 도착해서 차 안에서 한참 기다립니다.
- **인원 입력:** 웨이팅마다 인원을 직접 넣어야 합니다. 밥 먹고 한 명이 먼저 집에 가도 다음 가게 웨이팅 인원은 그대로입니다.
- **도착 인증:** 도착하면 앱에서 위치 인증을 하거나 코드를 따로 입력해야 합니다.
- **안전:** 위의 조작을 대부분 운전 중에 하게 됩니다.

2.2 기존 서비스가 이미 해결한 것과 남은 빈칸

기존 서비스	이미 제공하는 것	남은 빈칸
TMAP × 캐치테이블 (2024.6 제휴)	내비 앱에서 매장 검색 후 웨이팅 접수, 실시간 대기시간과 예상 이동시간 비교	운전자가 직접 보고 직접 접수. 인원은 수동. 도착은 알 수 없음
테이블링	원격 줄서기, 도착 인증(위치 인증 또는 대기확정코드), 순서 미루기(매장당 최대 3 회)	도착을 폰 위치나 코드 입력으로만 확인. 실제 몇 명이 왔는지는 모름
캐치테이블	원격 줄서기, 웨이팅 미루기(매장당 최대 2 회)	같음

ETA 와 대기시간을 비교해서 줄을 서는 기능은 이미 나와 있습니다. 그래서 이 부분은 내세우지 않고, 폰으로는 알 수 없는 탑승 인원, 도착, 하차 정보를 웨이팅에 쓰는 데 집중했습니다.

3. 앱 주요 기능 및 차량 특화 요소

3.1 핵심 컨셉: 차량이 증명하는 웨이팅

웨이팅 과정 중 세 시점에서 차량 신호를 씁니다.

순간	폰 앱	스마트 저니	사용 차량 신호
접수	인원수를 직접 입력	좌석 점유로 인원을 자동 제안	좌석 점유 (Seat)
이동 중	인원 변동을 반영하지 못함	다음 매장 출발 시 점유 인원으로 갱신 제안	좌석 점유 (Seat)
도착	위치 인증이나 코드를 직접 입력	도착 이벤트, 정차, 도어, 좌석 비움을 종합해 "N 명 중 M 명 하차 확인"을 매장에 자동 전송	내비 도착 이벤트, 주행 상태, 도어 (Door), 좌석 (Seat)

매장 입장에서는 접수한 팀이 실제로 왔는지, 몇 명이 내렸는지를 차량 정보로 받게 됩니다. 노쇼를 줄일 수 있고 인원에 맞춰 테이블을 미리 준비할 수 있습니다.

3.2 주요 기능

기능 1. 좌석 센서 기반 탑승 인원 자동 제안

좌석 점유 센서로 몇 자리에 사람이 앉아 있는지 읽어서 웨이팅 인원으로 제안합니다. 사람 수만 셀 수 있어서 성인과 아이는 구분하지 않고, 짐을 사람으로 잡을 수도 있어 정차 중에 바로 고칠 수 있게 했습니다. 센서를 못 읽으면 직접 입력합니다.

기능 2. 골든타임 알림과 원클릭·음성 승인 접수

내비의 남은 도착 시간(ETA)과 매장 예상 대기시간을 계속 비교하다가, 도착 직후에 호출이 오는 시점이 되면 큰 승인 카드와 음성으로 알려 줍니다. 운전자가 터치하거나 Gleo 에 말하면 접수되고, 승인 없이 자동으로 접수하지는 않습니다. 앰비언트 라이트 알림은 차량 제어 SDK 가 열리면 추가할 예정입니다.

- **알림 시점:** 남은 ETA 가 (예상 대기시간 − 도착 여유 10 분) 이하가 되면 알립니다. 매장 대기시간은 앞 팀 수 × 팀당 소요 시간으로 계산하고, 점주가 직접 바꿀 수도 있습니다.
- **예시:** 이동 60 분, 대기 30 분이면 출발 40 분 뒤(남은 ETA 20 분)에 알립니다. 이때 접수하면 호출은 30 분 뒤인데 차는 20 분 뒤 도착하니 10 분 정도 여유가 생깁니다.
- 대기가 짧아서 지금 걸어도 늦지 않으면 바로 알립니다. ETA 나 대기시간이 바뀔 때마다 다시 계산하고, 같은 웨이팅 알림은 한 번만 띄웁니다.

기능 3. 차량 신호로 도착·하차 확인

도착한 뒤 아래 조건이 모두 맞고 5 초 동안 그대로 유지되면 "N 명 중 M 명 하차"를 매장 POS 로 보냅니다.

1. 내비에서 목적지 또는 경유지 도착 이벤트가 온다.
2. 차가 멈춘 상태다. CarUxRestrictions 의 주행 규제가 풀린 상태를 기준으로 보고, 기어 값은 읽을 수 있을 때만 참고한다.
3. 문이 열렸다가 닫힌다.
4. 좌석 점유 수가 접수 인원보다 줄었다. 하차 인원은 접수 인원에서 남은 점유 수를 빼서 계산한다.

POS 에는 "도착, 3/4 명 하차"로 표시되고 호출은 점주가 직접 합니다. 운전자는 주차하느라 차에 남을 수 있어서 몇 명 중 몇 명이 내렸는지로만 보여 줍니다. 3 분 안에 확인이 안 되거나 센서를 못 읽으면 "도착했어요" 버튼으로 대신합니다. 내린 사람 수만 확인하고, 누가 내렸는지는 알지 못합니다.

기능 4. 정체·대기열 변동 대응과 코스 연결

정체로 도착이 많이 늦어지거나 매장 대기가 갑자기 줄면 순번 미루기나 다른 매장 먼저 가기를 카드와 음성으로 제안하고, 승인하면 내비 경로에 반영합니다. 방문지는 최대 3 곳(경유지 2 개 + 목적지)까지 영업시간과 대기시간을 보고 순서를 정해 내비에 한 번에 넣습니다. 순번 미루기는 기존 앱에도 있는 기능이라 보조 기능으로 분류했습니다.

3.3 차량 특화 요소 (폰 앱과 비교)

구분	폰 앱으로 가능한가	차량에서 달라지는 점
탑승 인원	불가. 직접 입력해야 함	좌석 점유 센서로 자동 제안
도착 시점 판단	부분 가능. 앱의 예상 이동시간 비교 수준	순정 내비의 안내 중인 경로·정체 기반 ETA 를 직접 사용
도착 확인	위치 인증 또는 코드 입력	도착 이벤트, 정차, 도어, 좌석 변화를 종합해 자동 확인
하차 인원	알 수 없음	좌석 점유 변화로 추정해 매장에 전달
주행 중 안전	조작 시 시선 분산	주행 중에는 입력 UI 잠금, 대형 단일 승인 카드와 음성만 허용
알림	폰 푸시	대형 카드와 음성 안내로 시선 이동 최소화 (앰비언트 라이트는 선택 기능)

3.4 주행 안전 원칙

- 주행 중에 쓸 수 있는 범위는 Pleos Connect 주행 규제 정책을 따릅니다. CarUxRestrictions 로 규제 상태를 확인해서, 규제 중에는 코스 설정·인원 수정·설정 화면을 막고 승인 카드와 음성만 남깁니다. 상태를 못 읽으면 주행 중으로 봅니다.
- 앱은 정차용 화면과 승인 카드 화면을 나누고, 승인 카드 화면만 Distraction Optimization(DO) 대상으로 잡을 계획입니다. 앱 카테고리(라이프스타일 또는 모빌리티 서비스)와 DO 허용 범위는 공식 기준을 확인해서 정하고, 허용이 안 되면 주행 중에는 음성 승인만 제공합니다.
- 접수, 순번 미루기, 경로 변경은 사용자가 한 번 승인해야 실행됩니다. 음성 승인과 화면 승인 규칙은 같습니다.
- 앰비언트 라이트는 선택 기능이고, 쓸 수 없으면 화면과 음성으로만 알립니다.

4. 주요 사용 시나리오

등장인물: 주말에 가족 4 명이 Pleos 차량을 타고 맛집 A 에서 밥을 먹고 카페 B 로 가는 상황입니다.

장면 1. 정차: 인원과 코스 설정

4 명이 타자 "오늘의 탑승 인원: 4 명"이 뜹니다. 차가 서 있을 때 동승자가 A 맛집과 B 카페를 고르면 앱이 방문 순서를 제안하고, 승인하면 내비에 바로 들어가 안내가 시작됩니다.

장면 2. 주행: 골든타임 승인

A 맛집까지 60 분, 대기는 30 분입니다. 40 분쯤 달려 남은 시간이 20 분이 되면 "지금 A 맛집 4 명 웨이팅 걸까요?" 카드와 음성 안내가 나옵니다. 운전자가 "응, 걸어 줘"라고 하면 접수되고 순번이 표시됩니다. 호출은 30 분 뒤에 오는데 차는 20 분 뒤에 도착합니다.

장면 3. 주행: 정체 대응

가는 길에 막혀서 15 분 늦어질 것 같으면 "도착이 15 분 늦어질 것 같아요. 순번 한 번 미룰까요?"라고 묻습니다. 승인하면 순번이 뒤로 밀리고 알림 시점도 다시 계산됩니다.

장면 4. 도착: 하차 확인과 입장

A 맛집 앞에 차를 세우고 문을 열자 좌석 점유가 4 에서 1(운전자)로 바뀝니다. 문이 닫히고 몇 초 지나면 앱이 "3/4 명 하차"를 매장에 보내고 점주 화면에 바로 뜹니다. 점주가 호출하면 일행은 번호표 QR 로 들어가고, 운전자는 주차하고 따라 들어갑니다.

장면 5. 이동: 인원 변동 반영

식사 후 한 명은 먼저 집에 가고 3 명이 다시 탑니다. 출발 전에 앱이 "B 카페 인원을 3 명으로 바꿀까요?"라고 묻고, 승인하면 B 카페 웨이팅이 3 명으로 바뀝니다.

5. 개발 계획

5.1 시스템 구성 (3-Tier)

캐치테이블, 테이블링 같은 웨이팅 서비스는 외부 API 를 열어 두지 않았습니다. 그래서 중계 서버와 매장용 POS 를 직접 만들어 처음부터 끝까지 실제로 동작하는 데모를 준비합니다.

- **차량용 앱 (Pleos Connect, Kotlin + Jetpack Compose):** 플레오스 내장 내비 뒤에서 도는 백그라운드 서비스가 앱의 본체입니다. 지도와 길 안내는 내장 내비가 그대로 맡고, 앱은 NaviHelper 로 목적지와 경유지를 넣은 뒤 남은 시간과 도착 이벤트를 받아 인원·도착·하차 확인과 골든타임 계산을 합니다. 운전자 화면에는 승인이 필요할 때만 카드와 음성이 뜨고, 코스 입력과 인원 수정 화면은 정차 중에만 엽니다. SDK 호출은 인터페이스로 감싸고 테스트용 가짜 구현을 같이 만들어, 에뮬레이터 신호가 불안정해도 같은 흐름을 시연할 수 있게 합니다.
- **클라우드 중계 서버 (Java 21, Spring Boot, WebSocket):** 접수·인원·도착 정보는 REST 로 받고, 대기열 변경과 호출은 WebSocket(STOMP)으로 차량과 POS 에 바로 보냅니다. 매장 대기시간 계산도 서버에서 합니다. 방문 순서 제안에 쓰는 카카오 또는 TMAP 경로 API 도 서버에서 호출해 API 키가 앱에 들어가지 않게 했고, 핵심 기능은 이 키 없이도 동작합니다. DB 는 PostgreSQL 을 씁니다.
- **매장 점주용 웹 POS (React, TypeScript, 태블릿):** 대기 팀 관리, 차량 ETA 와 도착·하차 표시, 호출 처리. 심사 때 POS 에서 대기 팀을 늘리면 차량 화면이 바로 바뀌는 장면을 보여 줄 수 있습니다.

5.2 SDK·API 사용 계획

용도	사용 SDK·API	비고
목적지·경유지 일괄 입력	NaviHelper requestRoute(), addWaypoint() (WaypointIndex FIRST·SECOND)	NAVI_ROUTE_SEARCH 권한. 경유지 최대 2 개. 경로 탐색이 끝났는지는 onRoutingCompleted()로 확인
경로 변경·재탐색	requestReRoute(), addWaypoint(), removeWaypoint()	결과는 NaviHelperEventListener 콜백으로 확인

용도	사용 SDK·API	비고
실시간 ETA	onTripInfo() 콜백 (보조: getRouteStateInfo() → onRouteStateInfo())	NAVI_ROUTE 권한. 조회 결과는 반환값이 아니라 콜백으로 받음. 남은 시간을 주는 필드는 에뮬레이터에서 먼저 확인
도착 이벤트	onDestinationArrived(), onWayPointArrived()	도착 확인의 시작 조건
탑승 인원	Vehicle SDK Seat getCurrentSeatOccupancy(area), registerSeatOccupancy, checkSeatOccupancyCapabilities	좌석 영역(SeatArea)별 점유 Boolean을 읽어 점유 수를 합산. 안전벨트 체결 상태(getCurrentSeatBeltBuckledState)로 짐 오탐과 하차 순서를 보조 확인. CAR_SEATS 권한 (Vehicle SDK 2.3.2 기준)
하차 확인(도어)	Vehicle SDK Door getCurrentDoorPosition, registerDoorPosition(area, listener), checkDoorPositionCapabilities	승객 도어(ROW_1~3) 영역별 열림 정도를 구독하고 보닛·트렁크는 제외. CAR_DOORS 권한 (Vehicle SDK 2.3.2 기준)
주행·정차 판별	AAOS CarUxRestrictionsManager (registerListener, onUxRestrictionsChanged, isRequiresDistractionOptimization). 보조로 기어 상태 조회	Pleos 공식 주행 규제 가이드의 방식. Vehicle SDK 의 기어 조회(getCurrentGearSelectionState)는 deprecated 이며 Google GEAR_SELECTION 경로로 이전하라는 안내가 있어 보조 조건으로만 사용. DrivingMode 는 주행 모드·크루즈 정보라 정차 판별에 쓰지 않음
앰비언트 알림 (선택)	Vehicle SDK Light setAmbientLightBrightnessColor, checkAmbientLightBrightnessColorCapabilities, getAmbientLightState	차량 제어(Vehicle Control) SDK 는 현재 "준비 중" 상태로 신청할 수 없어, 사용 가능해질 때만 적용. 없으면 화면·음성만 사용
음성 승인·조회	Gleo App Schema 함수 3 개 (optimizeDailyRoute, registerGoldenTimeWaiting, checkWaitingStatus)	assets/schema.json 제출 후 검수 승인 필요
코스 최적화	카카오모빌리티 다중 경유지 API 또는 SK TMAP API 중 하나	방문 순서 제안(보조 기능)에만 사용. Spring Boot 서버에서 호출해 소요시간 계산(API 키는 서버에만 보관). 키가 없어도 사용자가 고른 순서로 경로 입력 가능

5.3 도착·하차 확인 엔진의 규칙

1. 도착 이벤트, 정차, 문 열림·닫힘, 좌석 점유 감소가 모두 맞고 5 초 동안 유지돼야 확정합니다. 점유 값이 잠깐 튄 것만으로는 확정하지 않습니다.
2. 보내기 직전에 문과 좌석 값을 한 번 더 읽습니다.
3. 3 분 안에 확정이 안 되거나 센서를 못 읽으면 "도착했어요" 버튼으로 넘어갑니다. 확인하지 못한 도착은 POS 에 "인원 미확인"으로 따로 표시합니다.
4. 인원 수만 보내고, 누가 탔는지 같은 정보는 다루지 않습니다.

5.4 구현 전 검증 항목 (개발 첫 단계)

- SDK 버전을 먼저 고정하고, 에뮬레이터 Virtual Console 에서 좌석과 문을 조작했을 때 값이 제대로 들어오는지, NaviHelper onTripInfo()로 남은 시간이 들어오는지 확인합니다. 골든타임 계산이 이 값에 달려 있어 가장 먼저 봅니다.
- 정차 판별은 CarUxRestrictions 기준입니다. Virtual Console 에서 주행 상태를 바꿨을 때 규제 상태가 따라 바뀌는지 보고, 규제 해제가 P 단과 항상 같지는 않을 수 있어 애매하면 하차 확인을 미룹니다.
- 좌석(CAR_SEATS)·도어(CAR_DOORS) 조회와 NaviHelper 는 별도 신청 없이 쓸 수 있다고 공식 안내에서 확인했습니다. 차종마다 지원 여부가 다를 수 있어 check...Capabilities 로 먼저 확인합니다.
- 차량 제어(Vehicle Control) SDK 는 아직 준비 중이라 핵심 기능에서 뺐고, 앰비언트 조명은 SDK 가 열리면 붙입니다.
- 주행 중 승인 카드가 앱 카테고리와 DO 기준에서 허용되는지 확인합니다. 앱이 뒤에서 돌다가 승인 카드를 띄우는 방법(알림 전체화면 표시, 서비스에서 직접 화면 실행, Gleo 음성) 중 Pleos 에서 되는 것을 먼저 고릅니다.
- Gleo App Schema 검수 일정을 확인하고, 승인 전까지는 같은 흐름을 화면 승인으로 시연합니다. TextToSpeech SDK 도 준비 중이라 음성 응답은 Gleo 가 지원하는 범위에서만 만듭니다.
- 확인 안 된 신호는 성공 시나리오에 넣지 않고 다른 방법으로 시연합니다.

5.5 개발 단계

단계	기간	내용
Phase 1	11/15 까지	에뮬레이터 세팅, SDK 연동 확인, Spring Boot 서버 기본 API, 인원 제안 → 골든타임 → 승인 접수까지 동작, 시연 영상
Phase 2	11/16 – 12/2	Spring Boot WebSocket 서버와 매장 POS 연동, 도착·하차 확인, Virtual Console 로 좌석·문을 조작하고 POS 대기 팀을 바꿔 가며 전체 흐름 테스트
Phase 3	12/3 – 12/4	2 차 PT 라이브 데모. 심사위원이 POS 에서 대기 팀을 바꾸면 차량 화면이 바로 반응하는 것을 보여 줌
Phase 4	12 월 – 2027.2	네트워크 끊김, GPS 음영, 경유지 재출발 같은 예외 처리, UI 정리, 전시용 최종 APK

5.6 개인정보·데이터 최소화

- 밖으로 보내는 정보는 인원, 하차 인원, ETA(분), 도착 여부뿐입니다. 위치 좌표나 이동 경로는 보내지 않습니다.
- 처음 쓸 때 정차 상태에서 어떤 정보를 어디에 왜 보내는지 알려 주고 동의를 받습니다. 설정에서 언제든 철회할 수 있고, 동의 전에는 아무것도 보내지 않습니다.
- 점주는 접수 팀의 인원, ETA, 도착·하차 현황만 볼 수 있습니다.

6. 기대효과 및 발전 가능성

6.1 타깃 고객 분석

- **1 차 타깃:** Pleos Connect 가 들어간 차량(더 뉴 그랜저, 디 올 뉴 아반떼, 디 올 뉴 투싼)으로 주말에 두 곳 이상 들르는 운전자와 동승자. 아이와 함께 다니는 가족처럼 동승자를 챙길 일이 많은 사용자를 먼저 봅니다.
- **2 차 타깃(B2B):** 원격 줄서기 노쇼나 호출 취소로 손해를 보는 동네 인기 매장 점주. 차가 확인한 도착·인원 정보를 받으면 노쇼가 줄고 테이블을 미리 준비할 수 있습니다.

6.2 이해관계자별 가치

- **운전자·동승자:** 인원 입력이나 도착 인증을 따로 할 필요가 없고, 운전 중에는 승인 한 번이면 됩니다.
- **가맹점:** 예약금을 받지 않아도 실제로 온 손님과 인원을 확인할 수 있습니다.
- **Pleos 생태계:** 좌석·도어·내비 신호를 함께 쓰는 생활 서비스 사례가 하나 생깁니다.

6.3 예상 MAU 와 Retention (가정 기반 시나리오)

Pleos Connect 탑재 차량은 2026 년 10 월 기준 국내에 더 뉴 그랜저, 디 올 뉴 아반떼, 디 올 뉴 투싼 3 종이 나와 있습니다. 월별 판매 실적에서 확인된 판매 대수만 더하면 47,516 대입니다(그랜저 6·7·9·10 월 34,481 대, 아반떼 9·10 월 13,035 대). 판매 대수가 공개되지 않은 달과 10 월 말에 계약을 시작한 투싼은 넣지 않았고, 차종별 합계라 구형 재고가 일부 섞였을 수 있습니다. 아래는 이 숫자를 기준으로 잡은 목표 가정이며 검증 전 수치입니다.

단계	가정	결과 (판매 47,516 대 기준)
설치	탑재 차량 운전자 중 20%가 설치	약 9,500 대
월간 활성	설치 차량 중 50%가 월 1 회 이상 사용	MAU 약 4,750
재사용 (Retention)	첫 사용 후 30 일 내 재사용 목표 40%, 3 개월 내 25%	월 약 1,900 재사용

파일럿 운영(일부 매장, 에뮬레이터와 가능하면 실차)에서 아래 지표를 재서 가정을 고쳐 나갈 계획입니다.

6.4 성공 지표

- 도착 후 대기 0 분 즉시 입장 비율, 골든타임 알림 정확도(호출 전 도착 비율)
- 좌석 센서 인원 제안 수락률과 수동 수정 비율, 하차 확인 정확도
- 가맹점 노쇼율(스마트 저니 경유 웨이팅 대 일반 원격 줄서기)
- 단일 승인(원클릭·음성) 처리 비율과 주행 중 조작 횟수

6.5 수익 모델과 확장 계획

- **수익 모델:** 처음에는 동네 인기 매장에 웹 POS 를 무료로 깔아 드리고, 파일럿 결과를 보고 건당 수수료나 월 구독 중 하나로 정합니다. 요율은 아직 정하지 않았습니다.
- **가맹점 확보:** 인기 매장 몇 곳에서 먼저 시범 운영해 노쇼가 실제로 줄어드는지 보여 주고 넓혀 갑니다.
- **확장:** 전기차 충전소 대기 예약(Vehicle SDK 의 EV 배터리 정보로 충전 대기와 식당 웨이팅을 겹치는 방안), 호텔·백화점 발렛 출차 미리 준비, 드라이브스루에서 도착 시간에 맞춰 조리 시작.

6.6 한계와 위험

- 실제 상용 웨이팅 서비스와 연동하지는 않고 자체 서버와 POS 로 시연합니다. 상용 서비스와의 제휴는 다음 단계 과제입니다.
- 좌석 센서는 사람 수만 셀 수 있어 성인과 아이를 구분하지 못하고, 짐을 사람으로 잡을 수도 있습니다. 그래서 직접 고칠 수 있게 했습니다.
- 하차 확인은 인원 수 추정이라 신원 확인이나 보안용으로는 쓰지 않습니다.
- ETA 나 대기시간이 틀리면 호출 전에 도착하지 못할 수 있습니다. 계속 다시 계산하고 도착 여유 시간을 둬서 줄입니다.
- SDK 에서 경유지를 2 개까지만 넣을 수 있어 코스는 최대 3 곳입니다.
- 주행 중에 어디까지 쓸 수 있는지는 주행 규제와 심사 기준에 따라 달라질 수 있습니다. 그래서 주행 중에는 음성 위주로 쓰고, 화면 조작은 차가 서 있을 때 끝나도록 만들었습니다.

7. 참고자료 (선택)

- 화면 시안: 접수 이메일에 별첨 파일로 첨부 (제출 시 최신본으로 교체)
- 시연 영상: 핵심 루프 화면 녹화 (제출 시 링크 기재)
- GitHub 저장소: 소스 코드와 APK (제출 시 링크 기재)

기존 서비스 확인 출처:

- TMAP × 캐치테이블 제휴 기사: https://www.digitaltoday.co.kr/news/articleView.html?idxno=521834
- 테이블링 원격줄서기·도착 인증 안내: https://docs.channel.io/tabling/ko/articles/%ED%98%84%EC%9E%A5%EB%8C%80%EA%B8%B0%EC%9B%90%EA%B2%A9%EC%A4%84%EC%84%9C%EA%B8%B0-7a6e145b
- 웨이팅 미루기 기능 설명: https://twigtwig.tistory.com/entry/%EC%9C%A0%EC%9A%A9%ED%95%9C-%EC%95%B1%C2%B7%EC%9B%B9%EC%84%9C%EB%B9%84%EC%8A%A4-%EC%9B%A8%EC%9D%B4%ED%8C%85%EC%95%B1%EC%BA%90%EC%B9%98%ED%85%8C%EC%9D%B4%EB%B8%94%ED%85%8C%EC%9D%B4%EB%B8%94%EB%A7%81
- Pleos NaviHelper 경유지 제한: https://document.pleos.ai/en/docs/pleos-only/connect-sdk-pleos/NaviHelper/waypoint-settings
- Pleos Vehicle SDK Seat API (getCurrentSeatOccupancy): https://document.pleos.ai/docs/pleos-only/connect-sdk-pleos/Vehicle/vehicle/ai.pleos.playground.vehicle.api/seat/get-current-seat-occupancy
- Pleos Vehicle SDK Door API (registerDoorPosition): https://document.pleos.ai/docs/pleos-only/connect-sdk-pleos/Vehicle/vehicle/ai.pleos.playground.vehicle.api/door/register-door-position
- Pleos 주행 규제 개발 가이드 (CarUxRestrictions): https://document.pleos.ai/en/docs/pleos-only/vehicle-app-developer-guide/driving-restriction-development-guide

8. 숏폼 링크 (선택, 가산점 없음)

해당 없음 (제작 시 접수양식에 기재)